شرح سطر بسطر ١٧ — طبقة المحتوى: الأسئلة ومحتوى الدروس والنطق

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

كل محتوى الدروس (الرزمة، الشرح، الملخص، الفيديو، الكلمات، التدريب، الاختبار) ملفات JSON داخل `assets/content/lessons/<معرّف الدرس>.json`. هذه الطبقة تقرأها وتحوّلها إلى كائنات Dart مضبوطة الأنواع، فلا تتعامل الشاشات مع JSON خام أبدًا.

المبدأ: الجزء الغائب من الملف يعني «غير متوفر بعد» ولا نختلق بدله شيئًا. ودرس بلا ملف يظهر في القوائم بعلامة «قريبًا». ملفات المحتوى الحالية مؤقتة وتنتظر مراجعة معلم قبل الاعتماد.

النطق الصوتي (`SpeechService`) خلف واجهة مجرّدة ليُستبدل بنسخة مزيّفة في الاختبارات، وأي فشل فيه يُرجع false فتعرض الواجهة رسالة بدل أن ينهار التطبيق.

lib/features/content/question.dart — نماذج الأسئلة

lib/features/content/question.dart

ستة أنواع أسئلة مغلقة تحت `sealed class Question` ؛ فيكتشف المترجم أي `switch` ينسى نوعًا. التصحيح (`check`) داخل السؤال نفسه فيُختبر بلا واجهة.

الأسطر 1–5

```
/// نماذج الأسئلة التفاعلية. تُقرأ من ملفات محتوى الدروس (`assets/content/lessons`).
///
/// النصوص التي تحوي كسورًا تستعمل صيغة بسيطة: `{3/4}` كسر مرتّب، و`{?}` خانة فارغة
/// (تُرسَم بحدّ منقّط). يرسمها `MathText`.
library;
```

ماذا يفعل؟ تعليق المكتبة: يشرح صيغة النصوص الرياضية `{3/4}` للكسر و `{?}` للخانة الفارغة، ثم `library;` لتثبيت التعليق كتوثيق للملف.

لماذا بهذه الطريقة؟ صيغة بسيطة بدل HTML أو LaTeX: خفيفة، يكتبها مؤلف المحتوى بيده، ويرسمها `MathText` بودجتات حقيقية.

الأسطر 7–14

```
enum QuestionKind {
  choice,
  trueFalse,
  dragDrop,
  matchPairs,
  order,
  listenChoose,
}
```

ماذا يفعل؟ أنواع الأسئلة الستة: اختيار، صح/خطأ، سحب وإفلات، توصيل، ترتيب، استمع واختر.

لماذا بهذه الطريقة؟ enum يمنع الأخطاء الإملائية في أسماء الأنواع، ويُستعمل في الشارات وفي حفظ نتيجة كل سؤال.

الأسطر 16–26

```dart
/// عنصر مرئي يرافق السؤال: شريط كسر ملوّن، أو صف صور (تفاح…).
class QuestionVisual {
  const QuestionVisual.bars({required this.parts, required this.filled})
    : kind = 'bars',
      illustration = null,
      count = 0;

  const QuestionVisual.items({required this.illustration, required this.count})
    : kind = 'items',
      parts = 0,
      filled = 0;
```

ماذا يفعل؟ بداية `QuestionVisual`: العنصر المرئي المرافق للسؤال. مُنشئان مسمّيان: `bars` (شريط كسر) و`items` (صف صور). كلٌّ يملأ حقول الآخر بقيم صفرية.

لماذا بهذه الطريقة؟ صنف واحد بنوعين بدل صنفين: الحقول قليلة، ومكان استعماله (`QuestionVisualView`) يفرّق بحقل `kind`.

الأسطر 28–40

```dart
  factory QuestionVisual.fromJson(Map<String, dynamic> json) {
    return switch (json['kind']) {
      'bars' => QuestionVisual.bars(
        parts: json['parts'] as int,
        filled: json['filled'] as int,
      ),
      'items' => QuestionVisual.items(
        illustration: json['illustration'] as String,
        count: json['count'] as int,
      ),
      final other => throw FormatException('نوع مرئي غير معروف: $other'),
    };
  }
```

ماذا يفعل؟ قراءة الجزء المرئي من JSON حسب `kind`. نوع غير معروف يرمي `FormatException` برسالة واضحة.

لماذا بهذه الطريقة؟ الفشل المبكر الصريح أفضل من رسم خاطئ بصمت؛ والاختبار الذي يحمّل كل ملفات المحتوى يلتقط أي خطأ كتابة.

الأسطر 42–47

```dart
  final String kind;
  final int parts;
  final int filled;
  final String? illustration;
  final int count;
}
```

ماذا يفعل؟ حقول الجزء المرئي.

لماذا بهذه الطريقة؟ كلها `final`: الكائنات غير قابلة للتغيير.

الأسطر 49–58

```
sealed class Question {
  const Question({
    required this.id,
    required this.concept,
    required this.prompt,
    this.visual,
    this.hints = const [],
    this.solution = const [],
    this.explanation,
  });
```

ماذا يفعل؟ السؤال الأساسي (`sealed`) وحقوله المشتركة: المعرّف والمفهوم ونص السؤال والمرئي والتلميحات وخطوات الحل والشرح.

لماذا بهذه الطريقة؟ `sealed` يجعل الأصناف الستة هي الوحيدة الممكنة، فتكون عبارات `switch` على النوع شاملة بالضرورة.

الأسطر 60–74

```
  factory Question.fromJson(Map<String, dynamic> json) {
    final type = json['type'] as String;
    final base = (
      id: json['id'] as String,
      concept: json['concept'] as String,
      prompt: (json['prompt'] as String?) ?? '',
      visual: json['visual'] == null
          ? null
          : QuestionVisual.fromJson(json['visual'] as Map<String, dynamic>),
      hints: [for (final h in (json['hints'] as List<dynamic>? ?? [])) '$h'],
      solution: [
        for (final s in (json['solution'] as List<dynamic>? ?? [])) '$s',
      ],
      explanation: json['explanation'] as String?,
    );
```

ماذا يفعل؟ بداية `Question.fromJson`: يقرأ الحقول المشتركة في «سجل» (record) اسمه `base`. التلميحات والحل قوائم اختيارية فارغة افتراضيًا و`$h` يحوّل أي عنصر لنص.

لماذا بهذه الطريقة؟ نقرأ المشترك مرة واحدة ونمرّره لكل صنف، فلا نكرّر 7 حقول ست مرات.

الأسطر 75–91

```
    return switch (type) {
      'choice' => ChoiceQuestion._(
        base,
        choices: [for (final c in json['choices'] as List<dynamic>) '$c'],
        answerIndex: json['answer'] as int,
      ),
      'trueFalse' => TrueFalseQuestion._(
        base,
        statement: json['statement'] as String,
        answer: json['answer'] as bool,
      ),
      'dragDrop' => DragDropQuestion._(
        base,
        template: json['template'] as String,
        cards: [for (final c in json['cards'] as List<dynamic>) '$c'],
        answer: json['answer'] as String,
      ),
```

ماذا يفعل؟ تفريع حسب `type`: اختيار، صح/خطأ، سحب وإفلات؛ لكلٍ حقوله الخاصة.

لماذا بهذه الطريقة؟ مُنشئات الأصناف خاصة (`._`) فلا تُبنى الأسئلة إلا من JSON مفحوص.

الأسطر 92–107

```
      'matchPairs' => MatchPairsQuestion._(
        base,
        pairs: [
          for (final p in json['pairs'] as List<dynamic>)
            MatchPair.fromJson(p as Map<String, dynamic>),
        ],
      ),
      'order' => OrderQuestion._(
        base,
        items: [
          for (final i in json['items'] as List<dynamic>)
            OrderItem.fromJson(i as Map<String, dynamic>),
        ],
        startLabel: (json['startLabel'] as String?) ?? '',
        endLabel: (json['endLabel'] as String?) ?? '',
      ),
```

ماذا يفعل؟ التوصيل والترتيب: قوائم الأزواج والعناصر تُحوَّل بحلقة `for` داخل القائمة؛ وعنوانا طرفي الترتيب اختياريان.

لماذا بهذه الطريقة؟ الأزواج والعناصر مكتوبة بالترتيب الصحيح؛ الخلط يتم وقت العرض لا في الملف.

الأسطر 108–116

```
      'listenChoose' => ListenChooseQuestion._(
        base,
        word: json['word'] as String,
        choices: [for (final c in json['choices'] as List<dynamic>) '$c'],
        answerIndex: json['answer'] as int,
      ),
      _ => throw FormatException('نوع سؤال غير معروف: $type'),
    };
  }
```

ماذا يفعل؟ استمع واختر، ثم نوع مجهول يرمي `FormatException`.

لماذا بهذه الطريقة؟ ملف محتوى فيه نوع خاطئ يجب أن يفشل فورًا لا أن يُعرض سؤال ناقص.

الأسطر 118–138

```
  final String id;

  /// معرّف المفهوم في الكتالوج، منه يُحسب الإتقان.
  final String concept;
  final String prompt;
  final QuestionVisual? visual;

  /// تلميحات متدرّجة: تظهر واحدًا بعد كل إجابة خاطئة.
  final List<String> hints;

  /// خطوات الحل الكاملة (التلميح الأخير).
  final List<String> solution;

  /// جملة تظهر بعد الإجابة الصحيحة.
  final String? explanation;

  QuestionKind get kind;

  /// هل [answer] (انظر كل صنف) صحيحة؟ نوعها يطابق نوع السؤال.
  bool check(Object? answer);
}
```

ماذا يفعل؟ حقول السؤال المشتركة بتوثيقها، والدالتان المجرّدتان: `kind` و`check`.

لماذا بهذه الطريقة؟ `check(Object?)` موحّدة الشكل لكل الأنواع فيكتب المحرّك التصحيح مرة واحدة؛ ونوع الإجابة المتوقع موثّق عند كل صنف.

الأسطر 140–148

```dart
typedef _Base = ({
  String id,
  String concept,
  String prompt,
  QuestionVisual? visual,
  List<String> hints,
  List<String> solution,
  String? explanation,
});
```

ماذا يفعل؟ نوع «سجل» `_Base` يصف الحقول المشتركة.

لماذا بهذه الطريقة؟ يمرّره `fromJson` لمُنشئات الأصناف دون قائمة معاملات طويلة.

الأسطر 150–171

```dart
/// اختيار من متعدد. الإجابة: رقم الخيار (`int`).
final class ChoiceQuestion extends Question {
  ChoiceQuestion._(_Base b, {required this.choices, required this.answerIndex})
    : super(
        id: b.id,
        concept: b.concept,
        prompt: b.prompt,
        visual: b.visual,
        hints: b.hints,
        solution: b.solution,
        explanation: b.explanation,
      );

  final List<String> choices;
  final int answerIndex;

  @override
  QuestionKind get kind => QuestionKind.choice;

  @override
  bool check(Object? answer) => answer == answerIndex;
}
```

ماذا يفعل؟ سؤال الاختيار: الخيارات ورقم الجواب. `check` تقارن رقم الإجابة بـ `answerIndex`.

لماذا بهذه الطريقة؟ الإجابة برقم الخيار الأصلي لا بنصه، فيبقى التصحيح صحيحًا مهما اختلف ترتيب العرض بعد الخلط.

الأسطر 173–194

```dart
/// الإجابة المطلوبة: صح أو خطأ. `bool`.
final class TrueFalseQuestion extends Question {
  TrueFalseQuestion._(_Base b, {required this.statement, required this.answer})
    : super(
        id: b.id,
        concept: b.concept,
        prompt: b.prompt,
        visual: b.visual,
        hints: b.hints,
        solution: b.solution,
        explanation: b.explanation,
      );

  final String statement;
  final bool answer;

  @override
  QuestionKind get kind => QuestionKind.trueFalse;

  @override
  bool check(Object? a) => a == answer;
}
```

ماذا يفعل؟ صح/خطأ: العبارة والجواب المنطقي.

لماذا بهذه الطريقة؟ إجابة `bool` مباشرة.

الأسطر 196–222

```dart
/// اسحب البطاقة إلى الفراغ. القالب فيه `{?}`. الإجابة: نص البطاقة (`String`).
final class DragDropQuestion extends Question {
  DragDropQuestion._(
    _Base b, {
    required this.template,
    required this.cards,
    required this.answer,
  }) : super(
         id: b.id,
         concept: b.concept,
         prompt: b.prompt,
         visual: b.visual,
         hints: b.hints,
         solution: b.solution,
         explanation: b.explanation,
       );

  final String template;
  final List<String> cards;
  final String answer;

  @override
  QuestionKind get kind => QuestionKind.dragDrop;

  @override
  bool check(Object? a) => a == answer;
}
```

ماذا يفعل؟ سحب وإفلات: القالب فيه `{?}` والبطاقات والجواب نصًّا.

لماذا بهذه الطريقة؟ الجواب نص البطاقة لا موضعها، لأن البطاقات تُخلط وقت العرض.

الأسطر 224–237

```dart
/// طرف في سؤال التوصيل: نص/كسر، أو شريط كسر.
class PairSide {
  const PairSide({this.text, this.visual});

  factory PairSide.fromJson(Map<String, dynamic> json) => PairSide(
    text: json['text'] as String?,
    visual: json['visual'] == null
        ? null
        : QuestionVisual.fromJson(json['visual'] as Map<String, dynamic>),
  );

  final String? text;
  final QuestionVisual? visual;
}
```

ماذا يفعل؟ `PairSide`: طرف في التوصيل (نص أو شريط كسر).

لماذا بهذه الطريقة؟ طرف قد يكون كسرًا مكتوبًا أو شكلًا مرسومًا، فكلا الحقلين اختياري.

الأسطر 239–249

```dart
class MatchPair {
  const MatchPair({required this.left, required this.right});

  factory MatchPair.fromJson(Map<String, dynamic> json) => MatchPair(
    left: PairSide.fromJson(json['left'] as Map<String, dynamic>),
    right: PairSide.fromJson(json['right'] as Map<String, dynamic>),
  );

  final PairSide left;
  final PairSide right;
}
```

ماذا يفعل؟ `MatchPair`: زوج (يسار، يمين).

لماذا بهذه الطريقة؟ ثابت صغير يُقرأ من JSON.

الأسطر 251–276

```dart
/// الأزواج مكتوبة بالترتيب الصحيح. وصّل الأزواج (يسار i مع يمين i).
/// الإجابة: خريطة `Map<int, int>` من رقم العنصر الأيسر إلى رقم الأيمن
/// (بترتيب الأزواج الأصلي قبل الخلط).
final class MatchPairsQuestion extends Question {
  MatchPairsQuestion._(_Base b, {required this.pairs})
    : super(
        id: b.id,
        concept: b.concept,
        prompt: b.prompt,
        visual: b.visual,
        hints: b.hints,
        solution: b.solution,
        explanation: b.explanation,
      );

  final List<MatchPair> pairs;

  @override
  QuestionKind get kind => QuestionKind.matchPairs;

  @override
  bool check(Object? a) {
    if (a is! Map<int, int> || a.length != pairs.length) return false;
    return a.entries.every((e) => e.key == e.value);
  }
}
```

ماذا يفعل؟ التوصيل: التصحيح يشترط خريطة `Map<int,int>` كاملة وكل مفتاح يساوي قيمته.

لماذا بهذه الطريقة؟ الأزواج مكتوبة بالترتيب الصحيح، فالتوصيل الصحيح هو «العنصر i مع i». أي خريطة ناقصة أو من نوع آخر خطأ.

الأسطر 278–290

```dart
class OrderItem {
  const OrderItem({required this.label, this.visual});

  factory OrderItem.fromJson(Map<String, dynamic> json) => OrderItem(
    label: json['label'] as String,
    visual: json['visual'] == null
        ? null
        : QuestionVisual.fromJson(json['visual'] as Map<String, dynamic>),
  );

  final String label;
  final QuestionVisual? visual;
}
```

ماذا يفعل؟ `OrderItem`: عنصر قابل للترتيب (عنوان وشكل اختياري).

لماذا بهذه الطريقة؟ مثل `PairSide` لكن بعنوان إلزامي.

الأسطر 292–325

```dart
/// رتّب العناصر. العناصر مكتوبة بالترتيب الصحيح. الإجابة: قائمة أرقام العناصر
/// (`List<int>`) بالترتيب الذي اختاره الطالب.
final class OrderQuestion extends Question {
  OrderQuestion._(
    _Base b, {
    required this.items,
    required this.startLabel,
    required this.endLabel,
  }) : super(
         id: b.id,
         concept: b.concept,
         prompt: b.prompt,
         visual: b.visual,
         hints: b.hints,
         solution: b.solution,
         explanation: b.explanation,
       );

  final List<OrderItem> items;
  final String startLabel;
  final String endLabel;

  @override
  QuestionKind get kind => QuestionKind.order;

  @override
  bool check(Object? a) {
    if (a is! List<int> || a.length != items.length) return false;
    for (var i = 0; i < a.length; i++) {
      if (a[i] != i) return false;
    }
    return true;
  }
}
```

ماذا يفعل؟ الترتيب: التصحيح يقبل قائمة أرقام مرتّبة 0،1،2… فقط.

لماذا بهذه الطريقة؟ العناصر مكتوبة بالترتيب الصحيح؛ فالإجابة الصحيحة هي أن تعود أرقامها بالتسلسل.

الأسطر 327–353

```dart
/// استمع واختر. [word] تُنطق، و [choices] مفاتيح صور (apple…). الإجابة: رقم الخيار.
final class ListenChooseQuestion extends Question {
  ListenChooseQuestion._(
    _Base b, {
    required this.word,
    required this.choices,
    required this.answerIndex,
  }) : super(
         id: b.id,
         concept: b.concept,
         prompt: b.prompt,
         visual: b.visual,
         hints: b.hints,
         solution: b.solution,
         explanation: b.explanation,
       );

  final String word;
  final List<String> choices;
  final int answerIndex;

  @override
  QuestionKind get kind => QuestionKind.listenChoose;

  @override
  bool check(Object? a) => a == answerIndex;
}
```

ماذا يفعل؟ استمع واختر: الكلمة المنطوقة ومفاتيح الصور ورقم الجواب.

لماذا بهذه الطريقة؟ الخيارات مفاتيح رسوم (`apple` …) تُرسمها `Illustration` بالكود.

lib/features/content/lesson_content.dart — محتوى الدرس

`lib/features/content/lesson_content.dart`

كل جزء من الدرس صنف صغير غير قابل للتغيير، و`LessonContent` يجمعها ويعرف أي الأجزاء متوفر (`has`).

الأسطر 1–2

```dart
import '../../core/db/enums.dart';
import 'question.dart';
```

ماذا يفعل؟ استيراد قيم `LessonPart` وأسئلة الدرس.

لماذا بهذه الطريقة؟ نحتاج `LessonPart` لدالة `has` و`Question` لقوائم التدريب والاختبار.

الأسطر 4–16

```dart
/// كتلة في صفحة الرزمة: فقرة نص، أو عبارة رياضية، أو صورة (مكانها محجوز).
class PackBlock {
  const PackBlock({required this.type, required this.value});

  factory PackBlock.fromJson(Map<String, dynamic> json) => PackBlock(
    type: json['type'] as String,
    value: (json['value'] ?? json['caption'] ?? '') as String,
  );

  /// `text` أو `math` أو `image`.
  final String type;
  final String value;
}
```

ماذا يفعل؟ كتلة في صفحة الرزمة: نص أو عبارة رياضية أو صورة. `value` يأخذ `caption` للصورة إن لم يوجد `value`.

لماذا بهذه الطريقة؟ الصور الأصلية للرزمة غير متوفرة بعد؛ نحجز مكانها بعنوانها ولا نرسم ما لا نملكه.

الأسطر 18–37

```dart
class PackPage {
  const PackPage({
    required this.header,
    required this.title,
    required this.blocks,
  });

  factory PackPage.fromJson(Map<String, dynamic> json) => PackPage(
    header: (json['header'] as String?) ?? '',
    title: (json['title'] as String?) ?? '',
    blocks: [
      for (final b in json['blocks'] as List<dynamic>)
        PackBlock.fromJson(b as Map<String, dynamic>),
    ],
  );

  final String header;
  final String title;
  final List<PackBlock> blocks;
}
```

ماذا يفعل؟ صفحة الرزمة: رأس وعنوان وكتل. `fromJson` يحوّل قائمة الكتل.

لماذا بهذه الطريقة؟ الرأس والعنوان اختياريان في JSON وافتراضهما نص فارغ.

الأسطر 39–56

```
/// صف في رسم الشرح: كسر وشريطه (مثل ½ ثم ¼ …).
class BarRow {
  const BarRow({
    required this.label,
    required this.parts,
    required this.filled,
  });

  factory BarRow.fromJson(Map<String, dynamic> json) => BarRow(
    label: json['label'] as String,
    parts: json['parts'] as int,
    filled: json['filled'] as int,
  );

  final String label;
  final int parts;
  final int filled;
}
```

ماذا يفعل؟ صف في رسم الشرح: عنوان الكسر وعدد الأجزاء وعدد الملوّن.

لماذا بهذه الطريقة؟ تغذّي `FractionBar` مباشرة.

الأسطر 58–81

```
/// خطوة في «شرح مبسّط».
class ExplainStep {
  const ExplainStep({
    required this.title,
    required this.body,
    this.rows = const [],
    this.tip,
  });

  factory ExplainStep.fromJson(Map<String, dynamic> json) => ExplainStep(
    title: json['title'] as String,
    body: (json['body'] as String?) ?? '',
    rows: [
      for (final r in (json['rows'] as List<dynamic>? ?? []))
        BarRow.fromJson(r as Map<String, dynamic>),
    ],
    tip: json['tip'] as String?,
  );

  final String title;
  final String body;
  final List<BarRow> rows;
  final String? tip;
}
```

ماذا يفعل؟ خطوة شرح: عنوان ونص وصفوف أشرطة وملاحظة اختيارية.

لماذا بهذه الطريقة؟ خطوة بلا أشرطة أو ملاحظة تعرض النص فقط.

الأسطر 83–95

```dart
class SummaryPoint {
  const SummaryPoint({required this.text, this.math, this.mathNote});

  factory SummaryPoint.fromJson(Map<String, dynamic> json) => SummaryPoint(
    text: json['text'] as String,
    math: json['math'] as String?,
    mathNote: json['mathNote'] as String?,
  );

  final String text;
  final String? math;
  final String? mathNote;
}
```

ماذا يفعل؟ نقطة ملخص: نص وعبارة رياضية اختيارية وملاحظتها.

لماذا بهذه الطريقة؟ تُرسم النقطة بطاقة بها مربع رياضي صغير إن وُجد.

الأسطر 97–107

```dart
class VideoPart {
  const VideoPart({required this.title, required this.at});

  factory VideoPart.fromJson(Map<String, dynamic> json) =>
      VideoPart(title: json['title'] as String, at: json['at'] as String);

  final String title;

  /// وقت البداية `0:00`.
  final String at;
}
```

ماذا يفعل؟ جزء فيديو: عنوانه ووقت بدايته `0:00`.

لماذا بهذه الطريقة؟ وقت البداية نص كما يُكتب في الفيديو، لا نحوّله لأرقام.

الأسطر 109–135

```dart
/// بيانات الفيديو. [file] اسم الملف إن وُجد؛ بدونه يعرض المشغّل رسالة صادقة.
class VideoContent {
  const VideoContent({
    required this.title,
    required this.minutes,
    required this.sizeMb,
    required this.parts,
    this.file,
  });

  factory VideoContent.fromJson(Map<String, dynamic> json) => VideoContent(
    title: json['title'] as String,
    minutes: json['minutes'] as int,
    sizeMb: (json['sizeMb'] as num).toDouble(),
    parts: [
      for (final p in json['parts'] as List<dynamic>)
        VideoPart.fromJson(p as Map<String, dynamic>),
    ],
    file: json['file'] as String?,
  );

  final String title;
  final int minutes;
  final double sizeMb;
  final List<VideoPart> parts;
  final String? file;
}
```

ماذا يفعل؟ بيانات الفيديو: العنوان والدقائق والحجم والأجزاء، واسم الملف اختياري.

لماذا بهذه الطريقة؟ اسم الملف الغائب يعني أن المشغّل يعرض رسالة صادقة بدل أن يوهم بالتشغيل.

الأسطر 137–163

```dart
/// كلمة في درس الإنجليزية.
class WordItem {
  const WordItem({
    required this.en,
    required this.ar,
    required this.ipa,
    required this.illustration,
    required this.sentenceEn,
    required this.sentenceAr,
  });

  factory WordItem.fromJson(Map<String, dynamic> json) => WordItem(
    en: json['en'] as String,
    ar: json['ar'] as String,
    ipa: (json['ipa'] as String?) ?? '',
    illustration: json['illustration'] as String,
    sentenceEn: (json['sentenceEn'] as String?) ?? '',
    sentenceAr: (json['sentenceAr'] as String?) ?? '',
  );

  final String en;
  final String ar;
  final String ipa;
  final String illustration;
  final String sentenceEn;
  final String sentenceAr;
}
```

ماذا يفعل؟ كلمة في درس الإنجليزية: الكلمة وترجمتها ونطقها الصوتي ومفتاح صورتها وجملة المثال بلغتين.

لماذا بهذه الطريقة؟ تغذّي شاشة «الكلمات والنطق».

الأسطر 165–176

```dart
/// كل محتوى درس واحد. أي جزء غائب يعني أنه غير متوفر بعد لهذا الدرس.
class LessonContent {
  const LessonContent({
    required this.lessonId,
    this.pages = const [],
    this.steps = const [],
    this.summary = const [],
    this.video,
    this.words = const [],
    this.practice = const [],
    this.quiz = const [],
  });
```

ماذا يفعل؟ مُنشئ محتوى الدرس: كل قائمة افتراضها فارغة والفيديو `null`.

لماذا بهذه الطريقة؟ درس يحوي الشرح فقط مثلًا صالح؛ ما غاب غير متوفر.

الأسطر 178–195

```
  factory LessonContent.fromJson(Map<String, dynamic> json) {
    List<T> list<T>(String key, T Function(Map<String, dynamic>) f) => [
      for (final e in (json[key] as List<dynamic>? ?? []))
        f(e as Map<String, dynamic>),
    ];
    return LessonContent(
      lessonId: json['lessonId'] as String,
      pages: list('pack', PackPage.fromJson),
      steps: list('explanation', ExplainStep.fromJson),
      summary: list('summary', SummaryPoint.fromJson),
      video: json['video'] == null
          ? null
          : VideoContent.fromJson(json['video'] as Map<String, dynamic>),
      words: list('words', WordItem.fromJson),
      practice: list('practice', Question.fromJson),
      quiz: list('quiz', Question.fromJson),
    );
  }
```

ماذا يفعل؟ `fromJson` بدالة مساعدة عامة `list` تحوّل أي مفتاح قائمة إلى كائنات. المفتاح الغائب قائمة فارغة.

لماذا بهذه الطريقة؟ نكتب التحويل مرة واحدة لكل أنواع القوائم بدل تكرار الحلقة سبع مرات.

الأسطر 197–204

```
  final String lessonId;
  final List<PackPage> pages;
  final List<ExplainStep> steps;
  final List<SummaryPoint> summary;
  final VideoContent? video;
  final List<WordItem> words;
  final List<Question> practice;
  final List<Question> quiz;
```

ماذا يفعل؟ حقول المحتوى.

لماذا بهذه الطريقة؟ تبقى قوائم غير قابلة للتعديل.

الأسطر 206–214

```
  bool has(LessonPart part) => switch (part) {
    LessonPart.pack => pages.isNotEmpty,
    LessonPart.explanation => steps.isNotEmpty,
    LessonPart.summary => summary.isNotEmpty,
    LessonPart.video => video != null,
    LessonPart.words => words.isNotEmpty,
    LessonPart.practice => practice.isNotEmpty,
    LessonPart.quiz => quiz.isNotEmpty,
  };
```

ماذا يفعل؟ `has(part)` : هل الجزء متوفر؟ لكل جزء شرطه (قائمة غير فارغة، أو فيديو غير `null`).

لماذا بهذه الطريقة؟ مكان واحد يحدّد التوفر، وتعتمد عليه شاشة الدرس وخطة المراجعة وحساب نسبة التقدّم.

الأسطر 216–221

```
  /// الأجزاء المتوفرة بترتيب العرض.
  List<LessonPart> get parts => [
    for (final p in LessonPart.values)
      if (has(p)) p,
  ];
}
```

ماذا يفعل؟ `parts`: قائمة الأجزاء المتوفرة بترتيب العرض.

لماذا بهذه الطريقة؟ نسبة تقدّم الدرس = المنجز ÷ هذه القائمة، فلا يطالَب الطالب بجزء غير موجود.

lib/features/content/content_repository.dart — قراءة ملفات المحتوى

lib/features/content/content_repository.dart

الأسطر 1–5

```
import 'dart:convert';

import 'package:flutter/services.dart';

import 'lesson_content.dart';
```

ماذا يفعل؟ استيراد `dart:convert` لفكّ JSON وحساب البايتات، و`services` لـ `rootBundle`، ونموذج المحتوى.

لماذا بهذه الطريقة؟ كل ما يلزم لقراءة أصل وتحويله.

الأسطر 7–14

```
/// يقرأ محتوى الدروس من `assets/content/lessons/<معرّف الدرس>.json`.
/// (تعرض الواجهة ذلك بصدق) = محتواه غير متوفر بعد درس بلا ملف.
class ContentRepository {
  ContentRepository({
    Future<String> Function(String path)? loadString,
    Future<Iterable<String>> Function()? listAssets,
  }) : _loadString = loadString ?? rootBundle.loadString,
       _listAssets = listAssets ?? _manifestAssets;
```

ماذا يفعل؟ المستودع: توثيقه، ومُنشئ يقبل دالتي تحميل وسرد أصول اختياريتين، وإلا يستعمل `rootBundle` الحقيقي.

لماذا بهذه الطريقة؟ حقن الدالتين يجعل الاختبارات تقرأ الملفات بشكل متزامن؛ فالقراءة الحقيقية داخل الزمن الوهمي لا تكتمل أبدًا.

الأسطر 16–22

```
  static const folder = 'assets/content/lessons/';

  final Future<String> Function(String path) _loadString;
  final Future<Iterable<String>> Function() _listAssets;
  final Map<String, LessonContent?> _cache = {};
  final Map<String, int> _sizes = {};
  Set<String>? _ids;
```

ماذا يفعل؟ مجلد المحتوى وحقول الدوال وذاكرة مؤقتة للمحتوى والأحجام وقائمة المعرّفات.

لماذا بهذه الطريقة؟ كل ملف يُقرأ مرة واحدة ثم يُخدم من الذاكرة.

الأسطر 24–27

```
  static Future<Iterable<String>> _manifestAssets() async {
    final manifest = await AssetManifest.loadFromAssetBundle(rootBundle);
    return manifest.listAssets();
  }
```

ماذا يفعل؟ قراءة قائمة الأصول من `AssetManifest` الحقيقي.

لماذا بهذه الطريقة؟ الطريقة الرسمية لمعرفة الملفات الموجودة دون تثبيت قائمة في الكود.

الأسطر 29–39

```
  /// معرّفات الدروس التي لها ملف محتوى.
  Future<Set<String>> availableLessonIds() async {
    final cached = _ids;
    if (cached != null) return cached;
    final assets = await _listAssets();
    return _ids = {
      for (final a in assets)
        if (a.startsWith(folder) && a.endsWith('.json'))
          a.substring(folder.length, a.length - '.json'.length),
    };
  }
```

ماذا يفعل؟ معرّفات الدروس التي لها ملف: نأخذ أصول المجلد المنتهية بـ `.json` ونقصّ المسار والامتداد.

لماذا بهذه الطريقة؟ اسم الملف هو معرّف الدرس؛ فإضافة ملف محتوى جديد تكفي لتظهر «محمّلة» دون تعديل كود.

الأسطر 41–48

```
  /// حجم ملف محتوى الدرس بالبايت (0 إن لم يكن له ملف).
  Future<int> byteSize(String lessonId) async {
    final cached = _sizes[lessonId];
    if (cached != null) return cached;
    if (!(await availableLessonIds()).contains(lessonId)) return 0;
    final raw = await _loadString('$folder$lessonId.json');
    return _sizes[lessonId] = utf8.encode(raw).length;
  }
```

ماذا يفعل؟ حجم ملف الدرس بالبايت (UTF-8) مع تخزينه؛ 0 إن لم يوجد.

لماذا بهذه الطريقة؟ تعرضه شاشة التنزيلات صادقًا من الملف نفسه.

الأسطر 50–61

```
  /// محتوى درس، أو null إن لم يكن له ملف.
  Future<LessonContent?> load(String lessonId) async {
    if (_cache.containsKey(lessonId)) return _cache[lessonId];
    if (!(await availableLessonIds()).contains(lessonId)) {
      return _cache[lessonId] = null;
    }
    final raw = await _loadString('$folder$lessonId.json');
    return _cache[lessonId] = LessonContent.fromJson(
      jsonDecode(raw) as Map<String, dynamic>,
    );
  }
}
```

ماذا يفعل؟ تحميل محتوى درس: من الذاكرة إن وُجد، و`null` إن لم يكن له ملف، وإلا نقرأ ونحوّل ونخزّن.

لماذا بهذه الطريقة؟ التخزين يشمل نتيجة `null` أيضًا فلا نعيد السؤال عن ملف غير موجود.

lib/features/content/content_providers.dart — مزوّدات المحتوى

lib/features/content/content_providers.dart

الأسطر 1–4

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'content_repository.dart';
import 'lesson_content.dart';
```

ماذا يفعل؟ استيراد Riverpod والمستودع والنموذج.

لماذا بهذه الطريقة؟ المزوّدات تربط الشاشات بالمستودع.

الأسطر 6–8

```
final contentRepositoryProvider = Provider<ContentRepository>((ref) {
  return ContentRepository();
});
```

ماذا يفعل؟ مزوّد المستودع الوحيد.

لماذا بهذه الطريقة؟ يُستبدل في الاختبارات بنسخة تقرأ من الملفات مباشرة.

الأسطر 10–13

```
/// معرّفات الدروس التي لها محتوى (لتمييز «متوفر» من «قريبًا» في القوائم).
final contentIndexProvider = FutureProvider<Set<String>>((ref) {
  return ref.watch(contentRepositoryProvider).availableLessonIds();
});
```

ماذا يفعل؟ مزوّد معرّفات الدروس المتوفرة.

لماذا بهذه الطريقة؟ تعتمد عليه القوائم لتمييز «متوفر» من «قريبًا» بلا قراءة المحتوى.

الأسطر 15–21

```
/// محتوى درس بمعرّفه، أو null إن لم يتوفر.
final lessonContentProvider = FutureProvider.family<LessonContent?, String>((
  ref,
  lessonId,
) {
  return ref.watch(contentRepositoryProvider).load(lessonId);
});
```

ماذا يفعل؟ مزوّد `family` لمحتوى درس بمعرّفه، يعيد `null` إن لم يتوفر.

لماذا بهذه الطريقة؟ `family` يخزّن نتيجة كل درس على حدة ويشاركها بين شاشات الأجزاء.

lib/features/content/speech_service.dart — النطق الصوتي

lib/features/content/speech_service.dart

الأسطر 1–2

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_tts/flutter_tts.dart';
```

ماذا يفعل؟ استيراد Riverpod وحزمة النطق `flutter_tts`.

لماذا بهذه الطريقة؟ النطق يعمل بمحرّك الجهاز نفسه فلا يزيد حجم التطبيق ولا يحتاج إنترنت غالبًا.

الأسطر 4–16

```
/// النطق الصوتي (قراءة الشرح بالعربية، ونطق كلمات الإنجليزية).
/// يُرجع (لا محرّك نطق على الجهاز مثلًا) أي فشل false رسالة واجهة، فتعرض
/// ولا ينهار التطبيق.
abstract class SpeechService {
  /// ينطق [text] بلغة [language] (مثل `ar` أو `en-US`). [slow] نطق أبطأ.
  Future<bool> speak(
    String text, {
    required String language,
    bool slow = false,
  });

  Future<void> stop();
}
```

ماذا يفعل؟ واجهة `SpeechService` المجرّدة: `speak` بلغة وسرعة، و`stop`.

لماذا بهذه الطريقة؟ واجهة مجرّدة ليسهل استبدال المحرّك في الاختبارات ولا تعرف الشاشات تفاصيل المكتبة.

الأسطر 18–21

```
class FlutterTtsSpeechService implements SpeechService {
  FlutterTtsSpeechService() : _tts = FlutterTts();

  final FlutterTts _tts;
```

ماذا يفعل؟ التنفيذ الحقيقي ينشئ `FlutterTts` مرة واحدة.

لماذا بهذه الطريقة؟ كائن واحد مشترك بدل إنشاء محرّك مع كل نطق.

الأسطر 23–41

```
  @override
  Future<bool> speak(
    String text, {
    required String language,
    bool slow = false,
  }) async {
    try {
      await _tts.stop();
      final available = await _tts.isLanguageAvailable(language);
      if (available == false || available == 0) return false;
      await _tts.setLanguage(language);
      await _tts.setSpeechRate(slow ? 0.25 : 0.45);
      await _tts.setPitch(1.0);
      final result = await _tts.speak(text);
      return result == 1 || result == true;
    } catch (_) {
      return false;
    }
  }
```

ماذا يفعل؟ `speak` : يوقف أي نطق جارٍ، يتأكد أن اللغة متاحة (وإلا false)، يضبط اللغة والسرعة (0.25 بطيء و0.45 عادي) والنبرة، ثم ينطق. أي استثناء يعيد false.

لماذا بهذه الطريقة؟ محرّكات النطق تختلف بين الأجهزة وقد لا تحوي صوتًا عربيًا؛ نعيد false فتعرض الواجهة رسالة صادقة بدل انهيار.

الأسطر 43–49

```
  @override
  Future<void> stop() async {
    try {
      await _tts.stop();
    } catch (_) {}
  }
}
```

ماذا يفعل؟ `stop` يوقف النطق ويبتلع أي خطأ.

لماذا بهذه الطريقة؟ إيقاف الصوت عند مغادرة الشاشة لا يجوز أن يسبّب خطأً.

الأسطر 51–54

```
/// (محرّك النطق لا يعمل فيها) يُستبدل في الاختبارات بنسخة مزيّفة.
final speechServiceProvider = Provider<SpeechService>((ref) {
  return FlutterTtsSpeechService();
});
```

ماذا يفعل؟ المزوّد: يعيد التنفيذ الحقيقي.

لماذا بهذه الطريقة؟ الاختبارات تستبدله بـ `FakeSpeech` يسجّل ما طُلب نطقه.